

Objects in JavaScript Assignment

PW EARNERS : Coding and Web Development

By : Nishant Saini Sir

Topics Covered

- Objects
- Dot and Bracket Notation
- Object.keys(), Object.values(), Object.entries()
- Object Destructuring
- Shorthand Properties
- Spread Operator in Objects and Arrays
- Rest Parameters
- import and export
- Named and Default Exports
- Module Scope

Section 1 – Working with Objects

1. Create a User Object

Create an object named user containing the properties name, email, and role. Display the complete object.

Example:

Output:

```
{  
  name: "Rahul",  
  email: "rahul@example.com",  
  role: "developer"  
}
```

2. Access Object Properties Using Dot Notation

Create a product object containing name, price, and category. Use dot notation to display the product name and price.

Example:

Input:

```
const product = {  
  name: "Laptop",  
  price: 50000,  
  category: "Electronics"  
};
```

Output:

```
Laptop  
50000
```

3. Access Object Properties Using Bracket Notation

Create a user object containing name and email. Use bracket notation to display the email property.

Example:

Input:

```
const user = {  
  name: "Rahul",  
  email: "rahul@example.com"  
};
```

Output:

rahul@example.com

4. Dynamic Property Access

Store a property name in a variable and use bracket notation to access that property from an object.

Example:

Input:

```
const user = {  
  name: "Rahul",  
  email: "rahul@example.com"  
};
```

```
const key = "name";
```

Output:

Rahul

5. Update an Object Property

Create a user object with name and role. Update the role from "student" to "developer" and display the updated object.

Example:

Input:

```
{  
  name: "Rahul",  
  role: "student"  
}
```

Output:

```
{  
  name: "Rahul",  
  role: "developer"  
}
```

6. Add a New Property

Create a profile object containing name and email. Add a new property named `isLoggedIn` with the value `true`.

Example:

Input:

```
{  
  name: "Rahul",  
  email: "rahul@example.com"  
}
```

Output:

```
{  
  name: "Rahul",  
  email: "rahul@example.com",  
  isLoggedIn: true  
}
```

Section 2 – Object Keys, Values and Entries

7. Get Object Keys

Create a user object containing name, email, and role. Use `Object.keys()` to get all the property names.

Example

Input:

```
const user = {  
  name: "Rahul",  
  email: "rahul@example.com",  
  role: "developer"  
};
```

Output:

```
["name", "email", "role"]
```

8. Get Object Values

Create a product object containing name, price, and category. Use `Object.values()` to get all the values from the object.

Example:

Input:

```
const product = {  
  name: "Laptop",  
  price: 50000,  
  category: "Electronics"  
};
```

Output:

```
["Laptop", 50000, "Electronics"]
```

9. Get Object Entries

Create a settings object and use `Object.entries()` to convert its properties into key-value pairs.

Example:

Input:

```
const settings = {  
  theme: "dark",  
  language: "English",  
  notifications: true  
};
```

Expected Output:

```
[  
  ["theme", "dark"],  
  ["language", "English"],  
  ["notifications", true]  
]
```

10. Display Object Entries

Create an object containing a user's name and email. Use `Object.entries()` and `forEach()` to display each key along with its value.

Example:

Input:

```
const user = {  
  name: "Rahul",  
  email: "rahul@example.com"  
};
```

Output:

name: Rahul

email: rahul@example.com

Section 3 – Destructuring and Shorthand Properties

11. Object Destructuring

Create a user object containing name, email, and role. Use object destructuring to extract name and email into separate variables.

Example:

Input:

```
const user = {  
  name: "Rahul",  
  email: "rahul@example.com",  
  role: "developer"  
};
```

Output:

Rahul

rahul@example.com

12. Destructuring with Renaming

Create a product object containing name and price. Use destructuring to store the name property in a variable named productName.

Example:

Input:

```
const product = {  
  name: "Laptop",  
  price: 50000  
};
```

Expected Result:

productName = Laptop

13. Create an Object Using Shorthand Properties

Create variables named name, email, and role. Use shorthand property syntax to create a user object using these variables.

Example:

Input:

```
const name = "Rahul";  
const email = "rahul@example.com";  
const role = "developer";
```

Output:

```
{  
  name: "Rahul",  
  email: "rahul@example.com",  
  role: "developer"  
}
```

14. Destructure Function Parameters

Create a function named `displayUser` that receives a user object. Use object destructuring in the function parameters to access and display name and email.

Example:

Input:

```
displayUser({  
  name: "Rahul",  
  email: "rahul@example.com"  
});
```

Output:

```
Rahul  
rahul@example.com
```

Section 4 – Spread and Rest

15. Copy an Object Using Spread

Create a user object and use the spread operator to create a copy of it.

Example:

Input:

```
const user = {  
  name: "Rahul",  
  role: "developer"  
};
```

Expected Result:

```
const newUser = {  
  name: "Rahul",  
  role: "developer"  
};
```

16. Update an Object Using Spread

Create a user object containing name and role. Use the spread operator to create a new object and update the role to "developer".

Example:

Input:

```
const user = {  
  name: "Rahul",  
  role: "student"  
};
```

Output:

```
{  
  name: "Rahul",  
  role: "developer"  
}
```

17. Combine Two Arrays Using Spread

Create one array containing frontend technologies and another containing backend technologies. Use the spread operator to combine them into a single array.

Example:

Input:

```
const frontend = ["HTML", "CSS", "JavaScript"];  
const backend = ["Node.js", "Express"];
```

Output:

```
["HTML", "CSS", "JavaScript", "Node.js", "Express"]
```

18. Rest Parameters

Create a function named showSkills that accepts a developer's name as the first parameter and any number of skills using a rest parameter. Display the name and skills.

Example:

Input:

```
showSkills("Rahul", "HTML", "CSS", "JavaScript");
```

Output:

Name: Rahul

Skills: HTML, CSS, JavaScript